

Intelligent Dead Reckoning (IDR) System — Final Master Implementation Blueprint

SIH Problem Statement #26168 | ISRO / Department of Space | Theme: Smart Vehicles

AUTHORITATIVE FINAL VERSION

Merged from four AI-generated blueprint variants and reconciled against the current IDR project documentation and implementation plan.

Core targets: <10% positional drift during GNSS outage | ~10 Hz mobile | ~200 Hz edge with FOG IMU | no OBD/CAN | offline inference | Android-first

1. Document Authority, Status and Merge Policy

This document is the final consolidated master blueprint produced by comparing the four AI-generated blueprint files supplied for this project: the ChatGPT Master Blueprint, DeepSeek Master Blueprint, KIMI Master Blueprint, and Z AI Master Blueprint. It also preserves the governing decisions established in the project documentation and final planning/implementation plan already associated with the project.

The blueprints are highly consistent rather than mutually contradictory. The main differences are emphasis: the ChatGPT version is the most expansive on architecture, testing, security, and final-system criteria; KIMI is especially explicit about scope, dataset verification, architecture boundaries, and the screening-first strategy; DeepSeek adds a particularly clear dependency/sequencing/checklist treatment; Z AI compresses the same architecture into a concise four-track and UI-focused representation. This final version merges the strongest compatible elements without introducing unsupported requirements.

Authority rule: where an older statement conflicts with the newer implementation-plan direction, the newer project documentation and implementation plan win. In particular, the application track is treated as a genuinely polished Jetpack Compose product track, while Tracks B, C, and D may begin non-blocking foundational work in parallel with the screening-critical Track A.

2. Executive Project Definition

Intelligent Dead Reckoning (IDR) is a hybrid offline-trained, on-device-inferenced vehicle navigation system that maintains continuous positioning when GNSS becomes unavailable or unreliable. It combines smartphone inertial sensors, GNSS, classical strapdown INS, AI-based velocity estimation, Kalman-filter fusion, road-network map matching, and vehicle kinematic constraints. The final system targets two deployments: an Android phone-class application at approximately 10 Hz and an edge-deployable engine using higher-rate FOG-grade IMU data at approximately 200 Hz.

The project must demonstrate positional drift below 10% of distance traveled during GNSS-denied operation. Example competition scenarios include less than 5 m of drift over 50 m in under 1 minute or less than 100 m over 1 km at 60 km/h. The mobile implementation must not depend on OBD-II/CAN or cloud inference, and navigation must remain usable without a live internet connection by using pre-downloaded map data.

3. Non-Negotiable Product and Architecture Principles

- No vehicle hardware dependency for the mobile solution: no OBD-II, CAN bus, speedometer feed, or dedicated vehicle hardware.
- AI complements classical navigation. AI corrects, estimates, or adapts the INS/EKF system rather than replacing its mathematical foundation.
- Training is performed offline on desktop/cloud compute; final navigation inference runs locally on the phone or edge engine.
- Offline navigation is a core requirement. Demo-area road data should be pre-downloaded so navigation does not leak live location to tile services or fail in tunnels.
- One algorithmic navigation core is shared across mobile and edge deployments. Runtime and packaging differ; the navigation logic should not be independently reimplemented.
- All dataset assumptions that are not empirically confirmed must be treated as provisional until Phase 0 verification.
- The system must be evaluated with real held-out sequences and measured on-device performance, not only desktop results.

4. Competition Structure and Deliverables

Stage	Primary deliverable	Required evidence
Screening	Written proposal	Preliminary AI model(s), verified IO-VNBD handling, raw-INS baseline, AI-corrected trajectory plots, preliminary metrics, and a credible finale roadmap.
Finale	Working system	Full polished mobile app, edge engine, six core capabilities, benchmark validation against drift/update-rate targets, and evaluation on additional datasets supplied at that stage.

The immediate competition-critical path is Phase 0 → Phase 1 → Phase 2 → Phase 3. Track B, C, and D may start non-blocking foundational work from Day 1, but this parallel work must not starve the screening path.

5. Scope Boundaries

5.1. Screening Scope

- IO-VNBD data understanding, schema verification, loading, and a defensible train/test split.
- Classical strapdown INS mechanization baseline.
- First AI/ML correction model, focused on velocity/displacement correction and informed by the IO-VNBD authors' published approach.
- Ground truth vs raw INS vs AI-corrected trajectory plots on held-out test sequences.
- Short written proposal containing methodology, evidence, preliminary metrics, and finale architecture roadmap.

5.2. Finale Scope

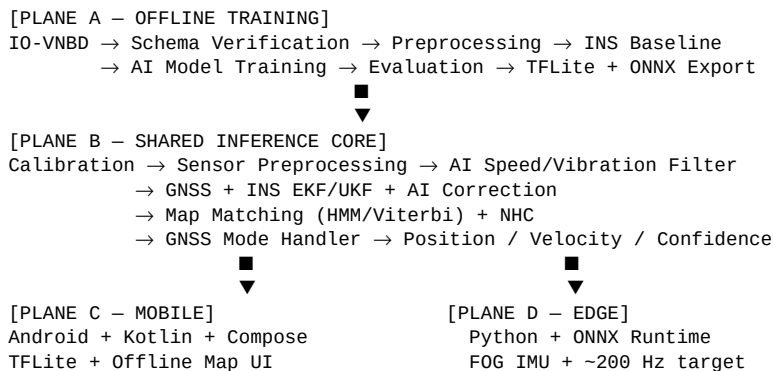
- In-vehicle alignment and calibration engine.
- AI speed and vibration filter refined for on-device inference.
- GNSS + INS fusion using an EKF/UKF baseline plus AI-enhanced correction/adaptive noise.
- Offline OSM map matching using HMM/Viterbi and non-holonomic constraints.
- Seamless GNSS deficit handling across Available, Degraded, and Denied states.
- Polished Android application with real-time map UI and animations.
- Edge-deployable high-rate engine with FOG-IMU ingestion and ONNX inference.
- Benchmarking, field testing, and final demo validation.

5.3. Explicitly Out of Scope

- OBD-II/CAN dependency
- iOS implementation
- Cloud-dependent inference during navigation
- Live-internet dependency for navigation/maps

6. System Architecture — Four Planes

Plane	Purpose	Major components
A — Offline Training Pipeline	Data, models, evaluation, export	IO-VNBD ingestion; preprocessing; INS baseline; model training; evaluation; TFLite/ONNX export
B — Shared Inference Core	Common navigation logic for both targets	Calibration; sensor preprocessing; exported models; GNSS+INS fusion; map matching; mode handling
C — Mobile Application	Android real-time product	SensorManager; GNSS APIs; TFLite; Compose UI; map renderer; pipeline orchestration
D — Edge Engine	Higher-rate deployment	FOG IMU ingestion; shared core; ONNX Runtime; position API/logging



7. Sensor and Signal Processing Pipeline

Sensor	Primary role	Failure/noise considerations
Accelerometer	Specific force → linear acceleration → velocity/displacement	Gravity contamination, bias, noise, vibration, potholes/braking
Gyroscope	Angular rate → orientation propagation	Bias drift, vibration, accumulated yaw error
Magnetometer	Absolute heading reference	Magnetic interference from engine/chassis/metal
GNSS	Absolute position/velocity when available	Multipath, degraded HDOP, blockage, complete tunnel loss

Shared preprocessing should include orientation-aware gravity removal, unit normalization, classical low-pass/band-pass vibration filtering ahead of the AI model, stationary-period detection for ZUPT opportunities, gyro-bias estimation, and magnetometer interference detection with down-weighting of implausible readings.

8. In-Vehicle Alignment and Calibration Engine

The phone-to-vehicle orientation is not assumed to be fixed. Calibration is therefore a first-class capability and must work for arbitrary mounting orientations.

8.1. Static Leveling

- Detect vehicle standstill using near-zero accelerometer variance or an equivalent verified stationary detector.
- Use the gravity vector in the phone frame to estimate pitch and roll.

8.2. Dynamic Yaw Alignment

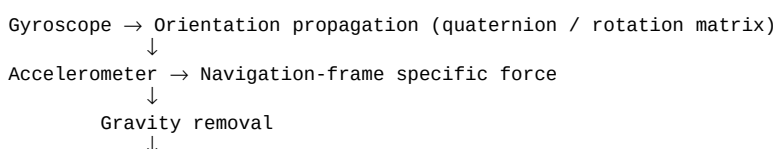
- Detect a short straight-driving segment with stable motion and available GNSS course.
- Compare phone-frame forward acceleration direction with vehicle course to solve the phone-to-vehicle yaw offset.

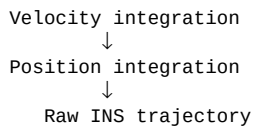
8.3. Continuous Recalibration

- Monitor pitch/roll/yaw estimates for discontinuities indicating phone pickup, movement, or remounting.
- Automatically rerun the calibration sequence when the trigger condition is satisfied.
- Once yaw has been established, the re-calibration trigger itself must not unnecessarily make the vehicle unusable during a repeated GNSS blackout.

9. Classical Strapdown INS Baseline

The classical INS is mandatory before AI correction. It is the mathematical scaffold and the screening "before" trajectory.





The raw INS trajectory is expected to drift significantly. That drift is valuable evidence because the screening submission needs an explicit comparison against the AI-corrected solution.

10. AI Speed and Vibration Filter

The first major AI model is a supervised sequence model trained against VBOX ground-truth velocity. It is intended to learn road-vibration and sensor-noise signatures and estimate forward velocity directly, reducing dependence on naive double integration.

10.1. Inputs and Windowing

- Gravity-corrected and calibration-aligned accelerometer data.
- Gyroscope data.
- Additional sensor/context features only when the verified dataset and experiments support them.
- Start with a 1–2 second rolling window (about 10–20 samples at 10 Hz) and tune experimentally.

10.2. Architecture

1D CNN → LSTM or GRU → Dense output

The CNN captures local vibration/noise patterns; the recurrent layer captures temporal dynamics; the dense output predicts forward velocity. An optional displacement-correction output may be retained only if evaluation supports it.

10.3. Training Target and Deployment

- Primary label: VBOX ground-truth velocity.
- GPS-derived ground speed may be used selectively during suitable GNSS-available segments.
- Export to TFLite for mobile and ONNX for edge deployment.
- Quantization may use float16 or int8 when needed to meet runtime constraints.

11. GNSS + INS Fusion Engine

The baseline fusion engine is an Extended Kalman Filter. A UKF may be evaluated if justified by implementation evidence, but the EKF is the default baseline because it is explicitly required and straightforward to validate.

Planned state vector: [position, velocity, orientation quaternion, accelerometer bias, gyroscope bias].

11.1. GNSS-Available Behavior

Use reliable GNSS fixes as measurement updates while continuing normal inertial propagation between fixes.

11.2. GNSS-Denied Behavior

When GNSS updates stop, the filter continues propagating from the AI-corrected INS state instead of switching to a separate navigation program.

11.3. AI-Enhanced Fusion

The final innovation layer should learn context-sensitive fusion behavior. Candidate mechanisms are learned adaptive process/measurement noise parameters or a learned residual correction applied to EKF outputs. The exact mechanism should be fixed only after the classical baseline and screening model are evaluated.

12. Map Matching and Kinematic Constraints

12.1. Offline Road Network

Extract the road network for the demonstration area from OpenStreetMap using OSMnx and store it locally.

12.2. HMM/Viterbi Map Matching

Represent candidate road segments as hidden states, treat the fused position as a noisy observation, and use Viterbi decoding to select the most plausible road path. The approach follows the standard HMM map-matching framing referenced by the project documents.

12.3. Non-Holonomic Constraints

Apply pseudo-measurements in the fusion model stating that lateral and vertical vehicle-frame velocities are approximately zero. This adds physical constraints that become particularly valuable when GNSS is denied.

12.4. Role in GNSS Outage

Map matching and NHC are drift-control mechanisms, not UI features. They should continuously constrain the solution and become especially influential during GNSS-denied operation.

13. Seamless GNSS Deficit Handler

GNSS quality is continuously classified as Available, Degraded, or Denied using indicators such as HDOP, satellite count, and signal quality. Optional map/context information may provide early warning before the signal fully disappears.

There is no crude hard switch. The filter continues to propagate inertially when GNSS measurements stop, reduces trust in degraded GNSS when necessary, and gradually re-converges when GNSS returns. The user experience should therefore be seamless even though a brief internal re-convergence window may be required.

14. Mobile Application Architecture

Platform: Android with Kotlin. UI framework: Jetpack Compose. Mobile inference: TensorFlow Lite. Map options: Google Maps SDK or an offline OSM renderer such as osmdroid.

Compose is the chosen framework because the UI must react continuously to streams of position/state updates and because the final app depends heavily on animation.

Component	Requirement
Vehicle marker	Eased interpolation between approximately 10 Hz position states; visually continuous motion
Camera	Smooth pan and heading rotation following the vehicle
GNSS state	Animated Available / Degraded / Denied indication tied to actual system state
Confidence/drift	Animated accuracy/confidence circle that grows during blackout and contracts on recovery
Calibration	Guided onscreen sequence with progress feedback
General polish	Material 3; day/night support; screen transitions; haptic feedback hooks

15. Jetpack Compose Development Track

Stage	Scope	Purpose
B1	Android Studio setup	Confirm emulator/device build and execution
B2	Kotlin fundamentals	Variables, functions, control flow, classes
B2.5	Compose fundamentals	State, recomposition, state hoisting, animation APIs
B3	Practice UI app	Animated state change
B4	Practice sensor app	Reactive sensor values
B5	Practice map app	Animated marker movement
B6	Design pass	Wireframes for map, calibration, system state, settings, trip summary
B7	App shell	Navigation and screen composition with placeholder data
B8	Core animated components	Marker easing, camera, status indicator, drift circle, calibration, transitions, theme
B9	TFLite tutorial	Load model, feed input, read prediction
B10	Final wiring	Replace placeholders with real Track A/C outputs

Marker easing, camera movement, screen transitions, and other animation mechanics can be built with placeholder data. Final tuning of the mode-status animation must wait for the real fusion engine because actual state flicker and debouncing behavior determine the correct animation strategy. This is a targeted tuning dependency, not a reason to block UI development.

16. Edge-Deployable Engine

The edge target uses Python with ONNX Runtime for development speed. NumPy/Numba optimization should be attempted before considering a full C++ rewrite. The engine is designed around higher-rate FOG-grade IMU input and an approximately 200 Hz target.

The internal loop must be rate-agnostic: algorithmic logic should scale from the approximately 10 Hz mobile use case toward approximately 200 Hz on the edge target without a conceptual redesign.

17. IO-VNBD Dataset Strategy

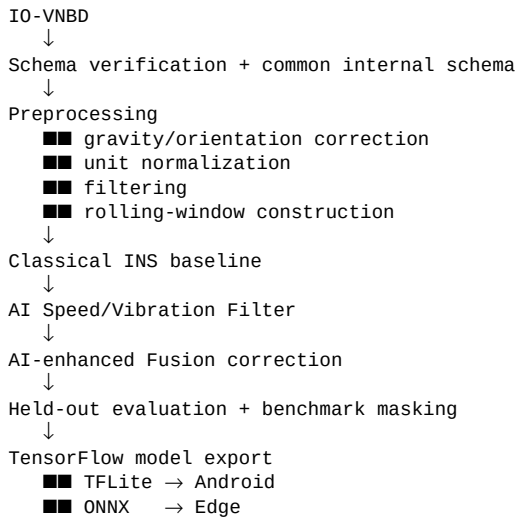
Primary dataset: IO-VNBD, using the "Synchronised V and S datasets" for screening and early development. The documented source is github.com/onyekpeu/IO-VNBD (Onyekpe et al., Data in Brief, 2020), CC BY licensed.

Attribute	Current project understanding
Data sources	Vehicle sensors + smartphone sensors; smartphone IMU/GNSS; VBOX reference data
Reference ground truth	Racelogic VBOX Video HD2
Phone sampling	Approximately 10 Hz
Coverage	UK, Nigeria, France
Scenarios	Traffic, roundabouts, hard braking, country roads, motorways
Approximate scale	~40 h / 1,300 km vehicle-extracted data; ~58 h / 4,400 km smartphone-recorded data
Known sensor issues	Documented acceleration and yaw-rate noise under vibration/braking
Published reference class	WhONet and related neural-network approaches reported substantial improvement over raw INS

Phase 0 must empirically verify exact CSV column names, units, timestamp semantics, synchronization tolerance, missing-data behavior, and the actual sampling/sub-rate of smartphone GPS fields. These items must not be assumed from the blueprint.

Screening strategy: use the Synchronised set only and select representative mixed-road sequences for training/validation/testing. The unsynchronised set should not add avoidable alignment complexity before it is needed.

18. Offline Training and Model Lifecycle



Training must remain reproducible through controlled seeds, documented data splits, explicit preprocessing versions, and saved model/configuration artifacts. Any preprocessing step used during training must have an equivalent, deterministic implementation for inference.

19. Technology Stack

Area	Technology	Role
Training language	Python 3.11	Primary ML/data environment
Data processing	pandas, NumPy, SciPy	CSV parsing, filtering, numeric transforms, windowing
ML training	TensorFlow / Keras	Model training and TFLite export path
Classical filter	FilterPy or custom NumPy	EKF/UKF baseline
Map matching	OSMnx + custom HMM/Viterbi	Offline road extraction and matching
Mobile	Android / Kotlin / Jetpack Compose	Product UI and device runtime
Mobile inference	TensorFlow Lite	On-device AI inference
Mobile maps	Google Maps SDK or osmdroid	Map visualization; offline option required
Edge	Python + ONNX Runtime	High-rate inference and portable deployment
Version control	Git	Repository and collaboration

20. Repository Architecture

```
idr-system/
├── data/
│   ├── raw/iovnbdd/          # gitignored dataset
│   └── processed/
├── training/
│   ├── preprocessing.py
│   ├── ins_mechanization.py
│   ├── speed_filter_model.py
│   ├── fusion_correction_model.py
│   ├── evaluate.py
│   └── export.py
├── core/                     # shared Plane B navigation logic
│   ├── calibration.py
│   ├── ekf_fusion.py
│   ├── map_matching.py
│   └── mode_handler.py
├── mobile/                   # Android / Kotlin / Compose
├── edge/                     # Python edge engine
├── models/                   # .tflite / .onnx artifacts
├── reports/
│   ├── data_schema.md
│   ├── screening_proposal.md
│   └── position_plots/
├── tests/
└── README.md
```

21. Evaluation and Benchmark Framework

Metric / test	Purpose
Primary: positional drift (m)	Direct competition metric during GNSS-denied intervals
Primary: drift % of distance	Normalizes error by traveled distance
Velocity RMSE / MAE	Measures AI velocity quality against VBOX ground truth
Orientation / heading RMSE	Measures attitude estimation quality
Trajectory comparisons	Ground truth vs raw INS vs AI-corrected/fused
Latency / throughput	Confirms 10 Hz mobile and progress toward 200 Hz edge
GNSS outage benchmark	Software-masked GPS stretches reproducing target scenarios
Recovery behavior	Measures smoothness/convergence when GNSS returns

Benchmark method: choose held-out sequences, deliberately mask GNSS for a defined stretch, run the complete pipeline, and measure the final position error against VBOX ground truth at the point where GNSS would have resumed. Repeat for representative speeds, road types, and outage conditions.

22. Testing Strategy

Test class	Required coverage
Unit tests	Quaternion integration, gravity removal, coordinate transforms, calibration triggers, EKF prediction/update against synthetic or hand-verifiable fixtures
Model tests	Fixed-seed reproducibility where applicable; input/output shape and range checks; exported-model round-trip tolerance
Integration tests	Raw IMU window → calibration → AI → fusion → map matching on real IO-VNBD sequence
On-device tests	Actual TFLite latency/throughput on a real Android device; confirm 10 Hz target
Edge performance	Measure high-rate ingestion, inference, and loop timing toward 200 Hz
Field test	Real drive through tunnel/underground area or controlled GNSS-masked route; compare displayed position against known reference

23. Security and Data Handling

- Raw GNSS/IMU data should not be transmitted off-device without an explicit purpose and consent where applicable.

- No cloud dependency is permitted for navigation inference.
- Offline map data should be stored locally for the demonstration/navigation region.
- Live location should not be unnecessarily sent to third-party tile servers during navigation.

24. Four-Track Development Model

Track	Focus	Key outputs
A — AI / Data Science	Dataset, preprocessing, INS, AI models, evaluation, export	Screening critical path + later model refinement
B — App Development	Android, Kotlin, Compose, map, animation, design system, TFLite	Polished mobile application
C — Algorithm Core	Calibration, EKF/UKF, AI fusion, map matching, mode handling	Shared navigation core
D — Edge Engine	Scaffold, high-rate ingestion, shared-core integration	Portable high-rate edge runtime

25. Integrated Timeline — 16 to 17 Weeks

Week	Track A	Track B	Track C	Track D
1	Dataset verification	Android setup + Kotlin	Calibration start	Edge scaffold
2	INS baseline	Kotlin + Compose start	Calibration + EKF start	Ingestion loop
3	AI training starts	Compose continuation	EKF continues	Ingestion loop
4	AI training continues	UI + sensor practice apps	EKF continues	—
5	AI training + export prep	Map practice app	Swap in AI input	—
6	Screening evidence target	Design pass	AI-enhanced fusion	—
7	Support C	App shell	Fusion + map work	Plug in Track C core
8	—	App shell + TFLite tutorial	Map matching	Core integration
9–11	—	Animated component build/tuning	Map + mode-switch validation	—
12–14	—	Full integration	Full integration	Full integration
15–16	—	Benchmark testing	Benchmark testing	Performance testing
17	—	Field test + demo prep	Field test + demo prep	Field test + demo prep

The updated plan expands Track B with dedicated Compose learning, design work, and animation tuning, which is the principal reason the total estimate increases to approximately 16–17 weeks.

26. Convergence Phases

26.1. Phase 6 — Full Integration

- Wire Track A models and Track C navigation core into the Android application.
- Tune mode-status and drift/confidence animations against real fusion outputs.
- Connect Track C core to the edge scaffold.

26.2. Phase 7 — Benchmark Testing

- Run systematic GNSS-denied simulations.
- Measure drift/update-rate/latency targets.
- Iterate on model/filter/map parameters when the 10% target is not met.

26.3. Phase 8 — Field Test and Demo Preparation

- Run a real tunnel, underground parking, or controlled GNSS-masked drive.

- Validate displayed position, transitions, marker smoothness, recovery behavior, and presentation readiness.

27. Detailed Phase Plan

27.1. Phase 0 — Dataset Acquisition and Schema Verification

- Acquire the Synchronised V and S dataset.
- Open representative files and inspect actual structure.
- Record column names, units, timestamps, sampling, synchronization, missing-data behavior, and GPS sub-rate.
- Document findings in reports/data_schema.md.

Completion condition: the actual schema is empirically confirmed and project assumptions are updated accordingly.

27.2. Phase 1 — Preprocessing and INS Baseline

- Implement preprocessing required by the verified schema.
- Implement gravity correction, orientation propagation, velocity integration, and position integration.
- Run a chosen test sequence and compare the raw INS path with VBOX ground truth.
- Create the first raw-INS trajectory plot.

Completion condition: a documented, visibly drifting baseline trajectory exists.

27.3. Phase 2 — AI Speed and Vibration Filter

- Create leakage-safe windows/splits.
- Train CNN + LSTM/GRU velocity model against VBOX labels.
- Evaluate on held-out sequences.
- Reconstruct trajectory using AI-corrected velocity.
- Quantify improvement over raw INS.

Completion condition: measurable improvement over the Phase 1 baseline is documented.

27.4. Phase 3 — Screening Proposal

- Package methodology and dataset evidence.
- Include raw INS and AI-corrected plots.
- Report preliminary metrics honestly.
- Describe the finale architecture and planned capabilities.

Completion condition: submission-ready screening proposal.

27.5. Phase 4 — Calibration Engine

- Static leveling.
- Dynamic yaw alignment.
- Continuous recalibration trigger and validation.

27.6. Phase 5 — EKF/UKF Fusion + AI Enhancement

- Implement and validate the classical EKF baseline.
- Feed AI-corrected motion into propagation.
- Add learned adaptive-noise/residual correction.

- Validate actual fusion-state behavior and GNSS-quality transitions.

27.7. Phase 6–8 — Convergence, Benchmark, Field

Proceed through integration, benchmark iteration, and field validation as defined in Sections 26 and 25.

28. Track Dependencies and Sequencing Rules

- Track A controls the screening critical path. The minimum acceptable sequence is Phase 0 → Phase 1 → Phase 2 → Phase 3.
- Track B may build the shell, map mechanics, animations, and practice apps with placeholder data from Day 1.
- Track B final mode-status animation tuning waits for Track C to expose real fusion-state behavior; this is not a block on earlier UI work.
- Track C can design and implement its calibration/fusion foundations in parallel, but schema-dependent details must wait for Phase 0 confirmation.
- Track D can build its scaffold and mock 200 Hz ingestion loop early; plugging in the real shared navigation core waits for Track C.
- No parallel track should consume enough time or people to jeopardize the screening evidence.

29. Master Checklist

Track	Completion checklist
A — AI/Data	Dataset verified; INS baseline plotted; AI model trained; evaluation documented; model exported
B — App	Android/Kotlin; Compose; 3 practice apps; design pass; app shell; animations; TFLite; final wiring
C — Core	Calibration; EKF/UKF; AI-enhanced fusion; map matching; NHC; mode handling; validation
D — Edge	Scaffold; ingestion loop; shared core plugged in; ONNX inference; rate/performance benchmark
Convergence	Full integration; benchmark passed; field test; demo-ready

30. Immediate Next Actions

- Track A: download the IO-VNBD Synchronised V and S dataset, open one real CSV in Python, document the actual schema, then implement Phase 1 INS baseline.
- Track B: install Android Studio, begin Kotlin basics, and start learning Compose animation fundamentals while the data team works.
- Track C: sketch calibration logic after the dataset schema is confirmed.
- Track D: create the Python edge skeleton and a mock 200 Hz data generator.
- First project action for everyone: do not hard-code preprocessing assumptions before the real CSV schema is inspected.

31. Canonical Phase 0/1 Starting Prompt

Using the IDR Final Master Blueprint, implement Phase 0 (Dataset Schema Verification) and Phase 1 (INS Mechanization Baseline). I have downloaded the IO-VNBD "Synchronised V and S datasets".

1. Load one representative CSV and print/document its actual columns, units, timestamps, sampling, missing-data behavior, and any synchronization details that can be established.
2. Update reports/data_schema.md from the actual evidence.
3. Implement the classical strapdown INS baseline in training/ins_mechanization.py using only assumptions confirmed by Phase 0. Include orientation propagation, gravity handling, velocity integration, and position integration.
4. Run the baseline on one test sequence and plot raw INS against VBOX ground truth.
5. Add basic tests for the mechanization math.
6. Stop after Phase 1. Do not implement the AI correction model yet.

32. Final System Success Criteria

Area	Definition of done
Data	Verified schema; reproducible preprocessing; documented splits
AI	Velocity model improves over raw INS on held-out sequences; exported models pass equivalence checks
Navigation	Calibration, INS, EKF, AI enhancement, map matching, NHC, and GNSS-state handling operate together
Mobile	~10 Hz position pipeline; smooth animated UI; offline map; polished calibration and state feedback
Edge	FOG IMU ingestion; ONNX inference; performance benchmark toward ~200 Hz
Benchmark	Drift measured against the <10% target and reported honestly across representative scenarios
Field	Real or controlled GNSS-denied drive completed and compared with an appropriate reference
Demo	Reproducible end-to-end demonstration with clear visualization of GNSS loss, DR continuity, and GNSS recovery

33. Source-Comparison Decisions and Reconciliations

Topic	Observed differences / emphasis	Final merged decision
Development strategy	Some versions stress screening-first; newer plan also allows parallel Track B/C/D starts	Use a screening-critical A path with non-blocking parallel B/C/D work from Day 1.
Mobile UI	All agree on Compose; newer plan adds dedicated learning/design/animation time	Polished Compose app is a first-class finale deliverable; animation mechanics can be built with dummy data.
Fusion AI	Some wording gives adaptive noise and others add residual correction as alternatives	Keep both as candidate mechanisms; select final mechanism after baseline/screening evidence.
INS baseline	All agree it must precede AI	Maintain classical INS as mandatory comparison/scaffold.
Dataset assumptions	All recent blueprints stress Phase 0 verification	Treat CSV schema, units, timing, and GPS sampling as unverified until measured.
Map matching	All converge on OSM + HMM/Viterbi + NHC	Treat map matching/NHC as active drift-control, not visualization-only.
GNSS switching	"instant/seamless switch" vs natural filter continuation	Implement continuous state degradation and recovery with no hard navigation switch.
Testing/security	ChatGPT version is more explicit	Retain unit/model/integration/on-device/field testing and local-data privacy controls.
Timeline	Newer plan explicitly expands to ~16–17 weeks	Use the expanded 4-track timeline and three convergence phases.

34. Architectural Invariants to Preserve During Implementation

- Never silently change the shared-core/mobile/edge separation without documenting the architectural reason.
- Never introduce OBD/CAN dependence into the smartphone path.
- Never rely on live cloud inference for navigation.
- Never claim benchmark compliance without measured GNSS-denied evaluation.
- Never use unverified IO-VNBD columns/units merely because they were listed in an earlier blueprint.
- Never let UI animation hide or invent system state; status visuals must reflect real fusion behavior.
- Never optimize for a demo trace at the expense of held-out and field robustness.

35. Document Governance

This blueprint is the working master specification for implementation, testing, documentation, and future planning until intentionally revised. After screening, Phase 4–8 details may be refined using screening feedback and the additional datasets/instructions provided by ISRO. Any such revision should preserve the validated architecture and explicitly record what changed, why it changed, and which benchmark evidence supports the change.

End of Final Master Blueprint.